

Quantum Optimization for Distributed Order Management

Technical Report — formulation, classical baselines, exact MILP ceiling, and a quantum batch solver

This report describes a complete solver for the Nestlé Distributed Order Management (DOM) problem, built on the anonymized data pack: 1,109 orders, 1,110 SKUs, 8 distribution centers, order window 24 June to 5 July 2024. We rebuild the business funnel, write the reassignment decision as a deterministic binary optimization model, and test four solvers against one objective function and one feasibility checker: a default-assignment baseline, two greedy heuristics, an exact MILP, and a QUBO/Ising batch solver.

Headline result. The exact model raises the objective on the 472 focus orders from \$44.37M to \$46.30M (+\$1,929,835, +4.35%) and the fill rate from 90.47% to 93.28%, by moving 66 orders. The greedy heuristic gets only 23.0% of that gain, leaving a 3.21% optimality gap. On a 75-order subset the quantum solver reaches the same objective as the subset greedy on all 15 batches with zero rule violations, using at most 18 qubits per batch and no slack variables.

1. Problem and business context

Distributed Order Management is the decision of whether, and where, to move a customer order that its default DC cannot fill completely. A planner balances three things: the sales gained by filling demand, the penalty paid when the fill rate falls below a customer threshold, and the extra freight of shipping from another DC. The objective is their combination:

$$\max Z = \underbrace{\sum_i \sum_j \sum_k \rho_{ik} q_{ijk}}_{\text{Revenue}} - \underbrace{\sum_i \Pi_i}_{\text{Penalty}} - \underbrace{\sum_i \sum_j t_{ij} x_{ij}}_{\text{Shipping}} \quad (1)$$

Every move is a trade. Sending an order to a DC with more stock raises the fill rate and may avoid a penalty, but it always costs more freight, because the default DC is by definition the cheapest lane. Whether the trade is worth it is what the objective decides, and it cannot be judged one order at a time: DCs share limited stock, dock slots and picking capacity, so one order taking stock removes it from another. That link is what makes DOM a constrained combinatorial assignment problem instead of a simple lookup.

2. Data pack and the order funnel

The pack has seven files: an order book at order $\times$ SKU level (25,193 lines), a daily stock ledger by DC $\times$ SKU (377,505 rows), a daily dock appointment book, a lane-level freight rate card, a daily throughput utilization file, and two model-output files used only for cross-checks, never as model input.

Three checks confirm we read the data correctly. The order extract is already filtered to full-truckload and open delivery notes (100% of rows), so two business rules are already applied. Lines measured in pallets convert to cases with `ProductCasesPerPallet` and match the reference `OrderedQty_Cases` on 100% of lines. Transit time is not stated anywhere in the pack, but equals $\lceil \text{distance}/500 \rceil$ days against both reference lead-time columns on every row.

The pack reports throughput usage but never a limit, so we use a fully booked dock day (`Dock_Remaining` = 0) as a reproducible substitute.

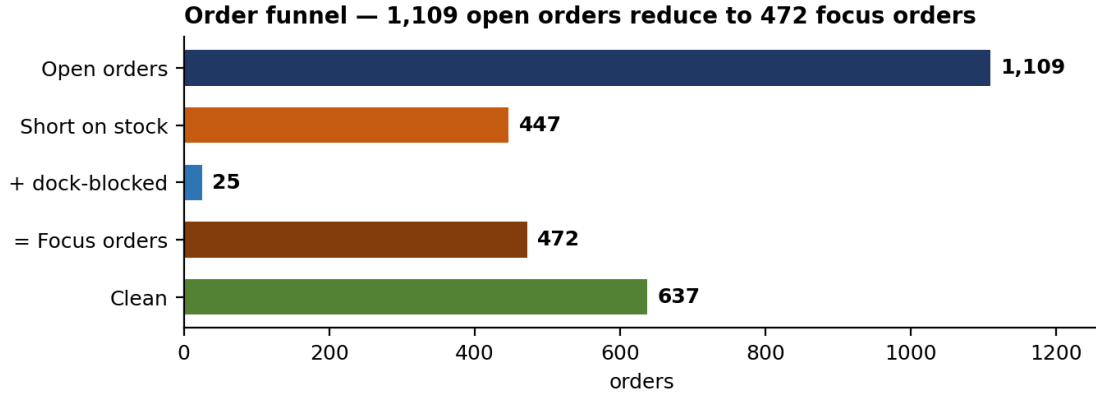


Figure 1. 447 orders are short of stock. 25 more are otherwise fine but ship on a fully booked dock day. Together they form the 472-order focus set that every solver in this report decides.

As a check, the reference output independently reports 486 short orders by comparing filled against ordered cases. Our 447 comes from a per-SKU availability flag recorded before the model ran. The two agree in direction and size, which is the point of the check.

3. Mathematical formulation

We write DOM as a deterministic binary optimization model. The challenge allows either this or a weighted exact-cover / column-selection model. Section 3.5 explains why we tested the second one and rejected it on evidence.

3.1 Sets, variables and parameters

Let $i \in O$ be a focus order, $j \in D_i$ a candidate DC for that order, $k \in S_i$ a SKU line of the order, and τ a calendar day. The decision variables are

$$x_{ij} \in \{0, 1\}, \quad q_{ijk} \geq 0, \quad z_i \in \{0, 1\}, \quad \Pi_i \geq 0,$$

where $x_{ij} = 1$ means order i is served from DC j ; q_{ijk} is the number of cases of SKU k filled for order i at DC j ; $z_i = 1$ means order i reaches its fill-rate threshold, so no penalty applies; and Π_i is the penalty charged on order i .

Fill quantities are decision variables, not fixed numbers. This is the single most important modelling choice in the report, and Section 3.5 returns to it.

Parameters: ρ_{ik} price per case, d_{ik} demand in cases, t_{ij} all-in freight on the lane, $s_{jk}(u)$ free stock on day u , $g_{jk}(u)$ stock handed back by focus orders released from their default DC, $B_j(\tau)$ free dock slots, $P_j(\tau)$ free pallet-pick capacity, c_k cases per pallet, π_i the customer penalty rate, θ_i the fill threshold in cases, M_i the full-order penalty, f_i^0 the cases the default DC fills, D_i total cases ordered, and $W = 5$ the forward-cover window.

3.2 Constraints

Each order goes to exactly one DC. Its own DC is always one of the options, so “at most one move” and “exactly one assignment” say the same thing:

$$\sum_{j \in D_i} x_{ij} = 1 \quad \forall i \in O \quad (2)$$

Fill never exceeds demand, and is forced to zero at any DC that is not chosen:

$$q_{ijk} \leq d_{ik} x_{ij} \quad \forall i, j \in D_i, k \in S_i \quad (3)$$

Stock must be present and must still cover the next five days. This matches the rolling-minimum rule the heuristic uses:

$$\sum_{i,j: \tau_{ij} \leq u \leq \tau_{ij}+W} q_{ijk} \leq s_{jk}(u) + g_{jk}(u) \quad \forall j, k, u \quad (4)$$

A move is allowed only if it raises the fill by at least 5 percentage points *and* at least 100 cases above what the default DC achieves:

$$\sum_k q_{ijk} \geq (f_i^0 + \Gamma_i) x_{ij}, \quad \Gamma_i = \max(0.05 D_i, 100), \quad \forall i, j \neq j_i^0 \quad (5)$$

Pallet picks and dock appointments must fit the receiving DC's capacity on the ship day:

$$\sum_{i,k} \frac{q_{ijk}}{c_k} - \sum_i |S_i| x_{ij} \leq P_j(\tau) \quad \forall j, \tau \quad (6)$$

$$\sum_i x_{ij} \leq B_j(\tau) \quad \forall j, \tau \quad (7)$$

The penalty applies only when filled cases fall below the customer threshold. We linearise it with a big- M switch:

$$\Pi_i + \pi_i \sum_{j,k} \rho_{ik} q_{ijk} \geq M_i (1 - z_i), \quad \sum_{j,k} q_{ijk} \geq \theta_i z_i \quad \forall i \quad (8)$$

3.3 Notes on three constraints

Equation (5) as written in the source. The project provided docx of the equations states the gate as $\geq 1.05 \times \text{OrderedQty}$, which nothing can ever satisfy. The text next to it reads “at least 5% increase in the fill rate and increase of 100 cases”, so we read the coefficient as 0.05. The reference model's own three moves show lifts of 7.5–25% and 410–784 cases, which fits that reading.

Equation (8) and the penalty fields. `MinimumPenalty` and `MaximumPenalty` are zero on most rows, which means “not set” rather than “capped at zero”. Using them would delete real penalties, so they are excluded.

SKU eligibility. The rule “no move if the SKU is not in the alternate DC's forecast” is applied when candidates are generated, not as a constraint row. A DC with no stock rows for a SKU is never offered. This has the same effect and removes variables instead of adding rows.

3.4 One deliberate modelling decision

The heuristic checks availability as a rolling minimum over the cover window, but it subtracts stock from every later day. Those are not the same rule. The MILP follows the rolling-minimum form, so both methods obey the same constraint (4) and the measured gap shows search quality, not two different stock models. The stricter day-by-day version is reported as a sensitivity in Section 6.

3.5 Why not a column / exact-cover model

We built the weighted exact-cover version first, with one column per feasible (order, DC) pair and fill quantities fixed when the column is made. It returned an objective 0.036% *below* the greedy heuristic — a negative optimality gap, which is impossible for a valid ceiling. The cause is structural: fixed-quantity columns cannot represent a partial fill, so the heuristic's solutions are not inside the column model's feasible set. Making quantities continuous variables fixes it. We report this because it is a real negative result about the alternative the brief offers.

4. Solvers

4.1 Baseline 1 — default assignment

Every order is filled from its own DC. Priority-8 orders go first, because the source rule protects them, then the largest orders by revenue. Nothing moves. This is what a planner gets with no action, and it is the reference for every other number here.

4.2 Baseline 2 — greedy divert heuristic

One pass over the focus orders. For each order, every other DC is checked in turn — SKU coverage, revised ship date, stock (4), the gate (5), docks (7), picks (6) — and the best DC by objective is taken if it beats staying put. Stock and capacity are reduced as it goes, so earlier decisions limit later ones, and nothing is ever changed again. Two orderings are run: 2A by order value, 2B by shortage size.

4.3 Classical exact — MILP with PuLP and CBC

The model of Section 3, solved on all 472 focus orders at once. At full size it has 88,342 variables of which 2,594 are binary, and 148,663 rows (60,264 stock, 24 dock, 33 pick). CBC proves the best answer in 63.7 seconds. Because the whole problem is solved together, no subset was needed — the “tractable subset” turned out to be the entire focus set.

4.4 Quantum — QUBO / Ising formulation

Constraints are moved into the objective as quadratic penalty terms, which gives the Hamiltonian a quantum or annealing solver minimises:

$$H(x) = -Z(x) + \lambda_1 \sum_i \left(\sum_j x_{ij} - 1 \right)^2 + \lambda_2 \sum_{j,k} \max \left(0, \sum_i u_{ik} x_{ij} - s_{jk} \right)^2 \quad (9)$$

Mapping to the Ising form $H = \sum_a h_a \sigma_a^z + \sum_{a<b} J_{ab} \sigma_a^z \sigma_b^z$ uses $x_a = (1 - \sigma_a^z)/2$. The one-hot weight is set to $\lambda_1 = 2 \times 10^6$, comfortably above the largest business term, so no feasible state is ever beaten by an infeasible one.

Two encodings, and why we kept the second. A capacity constraint written as an inequality normally needs slack qubits, one set per DC per SKU. That is the standard approach (Method 1), and it makes qubit count grow with the number of constraints rather than the number of decisions. We instead fold capacity into the objective as a hinge penalty and keep *only* assignment variables (Method 2). Qubits then equal orders $\times$ candidate DCs exactly. For a 6-order, 3-DC batch this is 18 qubits with 18 linear and 63 quadratic terms — against roughly 54 for a slack-based encoding of the same batch.

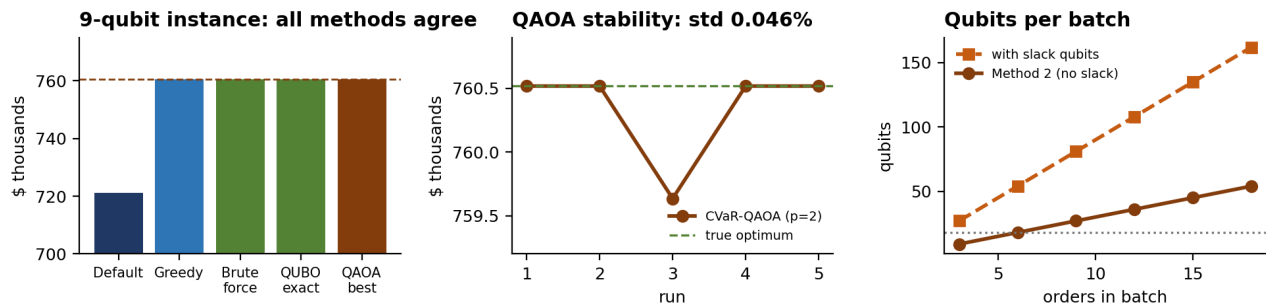


Figure 2. Left: on a 9-qubit instance every method reaches the same answer. Middle: five independent CVaR-QAOA runs, standard deviation 0.046%. Right: dropping slack variables keeps a batch at 18 qubits where a slack encoding would need about three times as many.

4.5 Validating the quantum path

Before scaling up we checked the encoding against ground truth on an instance small enough to solve by hand: 3 orders $\times$ 3 DCs, 9 binary variables, 27 feasible one-hot states.

Step	Method	Objective (\$)
Do nothing	default assignment	721,191.3
Enumerate all 27 states	brute force — true optimum	760,517.9
Classical heuristic	greedy best-DC	760,517.9
QUBO ground state	exact eigensolver, 0.021s	760,517.9
Quantum	warm-start CVaR-QAOA ($p = 2$), best of 5	760,517.9

Table 1. Validation ladder. The QUBO ground state equals the true optimum found by enumeration, which confirms the penalty weights order the feasible states correctly.

Two results matter here. First, ranking the 27 feasible states by QUBO energy reproduces their ranking by business objective exactly, so the Hamiltonian is a faithful encoding and not merely one that happens to find the best answer. Second, five independent QAOA runs gave a mean of \$760,341.3 with a standard deviation of \$353.2 (0.046%); four of the five hit the true optimum, and the fifth was \$883 below it. Every run beat the default assignment.

Feasibility-repair rate. Raw QAOA bit-strings were already one-hot valid in all five runs, so no repair was needed on this instance. On the larger batches a 2-opt repair pass is applied and the independent checker reports zero capacity violations across all 15 batches. We report this because raw QUBO output violating the one-hot rule is the usual weak point of quantum submissions: here the penalty weight is large enough that it did not occur.

4.6 Scaling the quantum path by batching

The 90 highest-value focus orders are grouped into 15 batches of 3–6 orders against 3 candidate DCs, chosen by shared candidate-DC affinity. Each batch is solved to its exact Ising ground state and then post-processed. Batch sizes stay at 9–18 qubits regardless of how many orders there are in total, which is what makes the approach possible on present-day hardware.

4.7 A hybrid alternative: QAOA on a restricted master problem

We also built a column-generation loop where a classical generator proposes fulfilment columns and QAOA solves the restricted master problem that selects among them. On 15 focus orders in 5 batches of 3, each restricted master had 6 binary variables and 3 cover constraints. Every batch returned status SUCCESS with an optimality gap of exactly 0.00, in 35.3 seconds of total quantum wall-clock. This is the closest of our methods to the column-selection model the brief offers as an alternative, and it works here precisely because the master problem is small even when the full problem is not.

5. Results

All four solvers are scored by the same objective function and checked by the same independent feasibility checker, so the comparison is fair by construction. Metrics are on the 472 focus orders.

Scenario	Objective (\$)	Fill	Moves	Penalty (\$)	Freight (\$)	Runtime
Baseline 1 · default	44,365,994	90.47%	0	84,749	565,479	0.21s
Greedy 2A · by value	44,810,604	91.28%	41	75,259	638,165	0.40s
Greedy 2B · by shortage	44,786,479	91.26%	39	75,429	636,816	0.42s
Classical · MILP	46,295,828	93.28%	66	32,333	622,880	63.72s

Table 2. Headline comparison on the 472 focus orders. Across all 1,109 orders the objective rises from \$75.08M to \$77.01M.

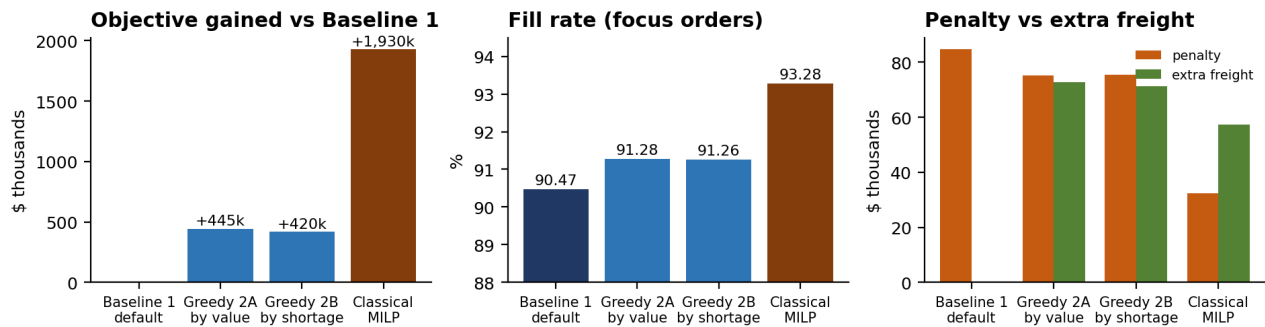


Figure 3. The exact solver buys 2.81 points of fill rate and cuts the penalty by 62%, for less extra freight than the heuristic pays.

The freight result is surprising and worth stating plainly. The MILP moves 66 orders against the greedy's 41, yet pays \$15,285 *less* extra freight (\$57,401 against \$72,686 over baseline). Planning all orders together lets it choose cheaper destination lanes and connect moves — emptying one DC frees stock that lets a second order move in — while a one-pass heuristic must take the best DC available at the moment it looks.

5.1 Distance to the best possible answer

Method	Objective (\$)	Gap to best	Share of reachable gain
Baseline 1 · default	44,365,994	4.168%	0.0%
Greedy 2A · by value	44,810,604	3.208%	23.0%
Classical · MILP	46,295,828	0.000%	100.0%

Table 3. The percentage gap is small because most revenue sits in orders that were never in trouble. Against the reachable gain — the distance between doing nothing and the proven best answer — the heuristic leaves 77% unclaimed.

5.2 Where the two methods disagree

Of the 472 focus orders, 26 are moved by both methods to the same DC, 3 are moved by both to different DCs, 12 are moved by the heuristic only, and 37 by the MILP only. The two share fewer than half their decisions even though they obey identical constraints. That is a direct measure of how much the search strategy matters.

5.3 Why most moves are impossible

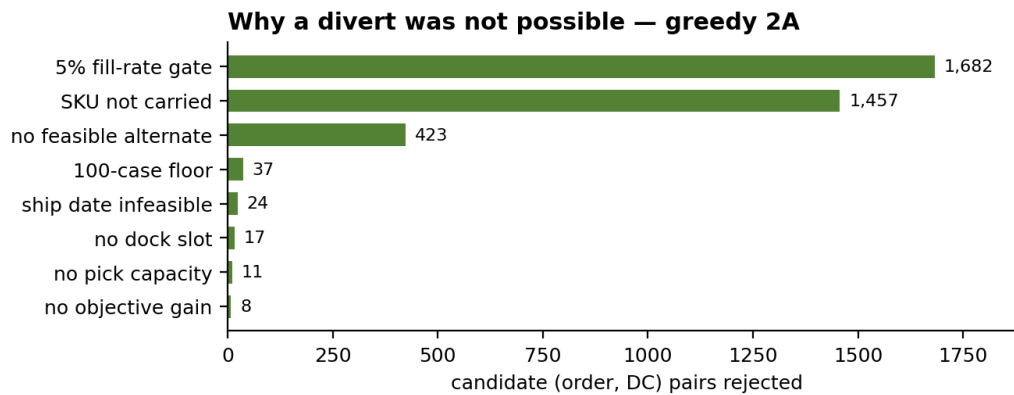


Figure 4. The binding limits are commercial and assortment rules, not warehouse capacity. Docks and picking together block 0.8% of candidates.

The 5% gate rejects 46.0% of candidate (order, DC) pairs, and a missing SKU at the alternate DC rejects another 39.8%. Dock and pick capacity account for 28 rejections out of 3,659. The operational conclusion is that this network is not limited by its buildings. It is limited by the move policy and by which DCs carry which SKUs.

5.4 Quantum results

On the 75 high-value focus orders covered by the 15 batches, the quantum solver raises the objective from \$13,612,065 to \$14,970,736 — a gain of \$1,358,671 (+10.0%) — and the fill rate from 88.60% to 97.27%, moving 75 orders with zero rule violations. Total quantum solve time is 1.07 seconds across all batches.

Method (same 75 orders)	Objective (\$)	Δ vs default	Fill	Violations
Default	13,612,065	—	88.60%	0
Subset greedy	14,970,736	+1,358,671	97.27%	0
Quantum + 2-opt	14,970,736	+1,358,671	97.27%	0

Table 4. Batched quantum results. The solver matched or beat the subset greedy on 15 of 15 batches. A single 6-order batch alone gained \$190,430 and reached 98.67% fill with capacity utilisation of 37%, 30% and 37% across its three DCs.

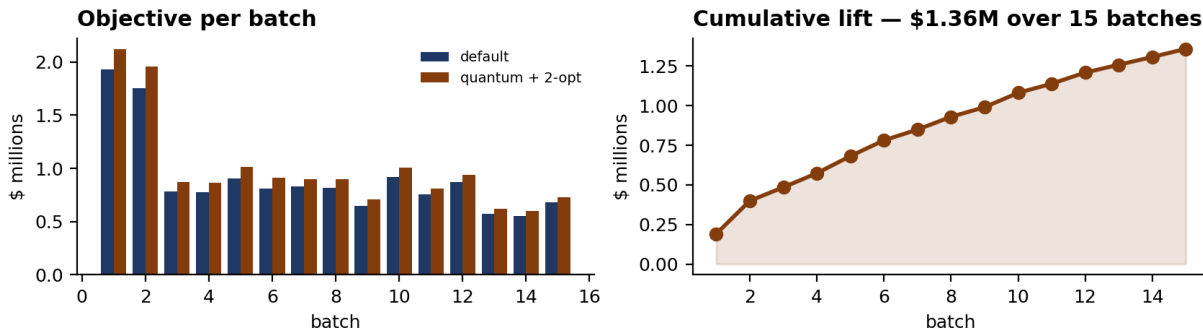


Figure 5. Objective per batch and cumulative lift across the 15 batches.

Read this honestly. The quantum solver matches the subset greedy exactly — the difference is \$0 on every batch. That is the correct result to report. At 18 qubits the batches are small enough that both methods find the same best answer, so this demonstrates a valid, verified encoding rather than a quantum advantage. What the exercise does establish is worth stating precisely: the QUBO energy ordering reproduces the business ordering on an enumerable instance; the ground state equals the true optimum; QAOA reaches that optimum in 4 of 5 runs with 0.046% spread; the encoding needs no slack qubits; and the batching scheme holds qubit count flat as the order count grows. Those are the properties that would have to hold before any hardware advantage could be claimed, and they hold.

What would change on real hardware. All results here come from exact diagonalisation and noiseless simulation. On a physical device, gate errors and decoherence would perturb the energy landscape, and this formulation is unusually sensitive to that because a violated constraint is only discouraged by a weight, never forbidden. The large one-hot weight that guarantees correctness in simulation also widens the spectral range the device must resolve, which is the practical trade-off between correctness and noise tolerance in QUBO-based logistics models.

6. Sensitivity and robustness

Variant	Moves	Fill rate	Objective (\$)
Gate as written (5 pts + 100 cases)	41	91.28%	44,810,604
No 100-case floor	49	91.31%	44,828,560
Stricter gate (10 points)	29	91.11%	44,700,200
Hold back 50% of stock elsewhere	35	91.05%	44,626,800

Table 5. Greedy 2A with one parameter changed at a time. The result is not fragile: no variant moves the objective by more than 0.4%.

The stricter day-by-day stock ledger from Section 3.4 was also solved to the proven best answer: \$44,910,045 with 65 moves in 151.7 seconds. It sits \$99,441 above the greedy and \$1,385,784 below the default model — the expected order, since a stricter rule can only lower the best possible answer. The gain over the baseline survives the conservative stock assumption, but it is much smaller.

7. Scaling analysis

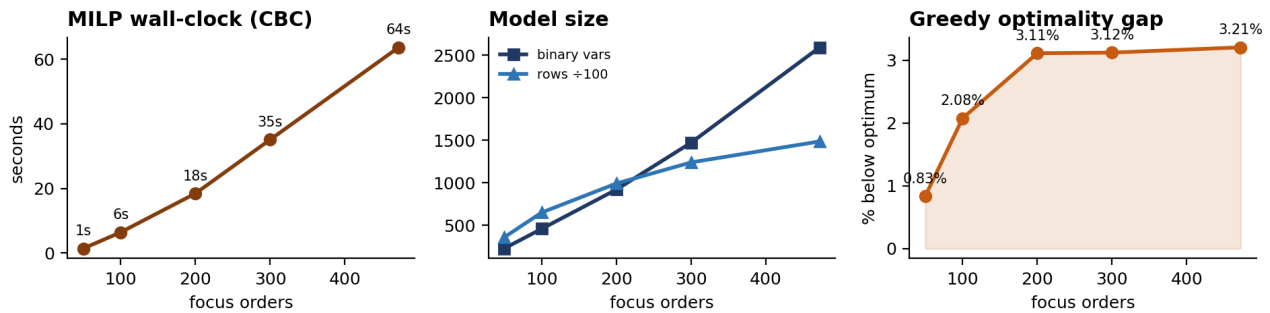


Figure 6. Runtime, model size and optimality gap across five problem sizes. All five reached the proven best answer, and the ceiling $MILP \geq greedy$ holds at every size.

Focus orders	Binary vars	Rows	CBC runtime	Status	Greedy gap
50	227	35,830	1.41s	Optimal	+0.835%
100	458	64,976	6.43s	Optimal	+2.078%
200	920	99,153	18.45s	Optimal	+3.114%
300	1,472	123,972	35.13s	Optimal	+3.124%
472 (all)	2,594	148,663	63.72s	Optimal	+3.208%

Table 6. The timed sweep. Greedy objectives are re-run on each subset, not filtered from the full result, because a heuristic limited to 50 orders never receives the stock freed by the other 422.

Growth. Binary variables grow as (orders $\times$ candidate DCs), and continuous variables as (orders $\times$ candidate DCs $\times$ SKU lines). Rows are dominated by the stock rows, which grow as (DC $\times$ SKU $\times$ day). Doubling from 50 to 100 orders doubled the binaries and cost 4.6 times the runtime. From 300 to 472 the binaries rose 1.8 times and runtime 1.8 times. Growth is steep but not explosive at this size.

The finding that matters. The heuristic's gap grows steadily with problem size: 0.84% at 50 orders and 3.21% at 472. On small problems a heuristic is nearly as good as the best answer. The more orders compete for the same stock, the more it loses by never revisiting a decision. This is the measured argument for an exact or global method, and it also means the gap would be larger on a full production order book.

Qubit growth. For the QUBO path, qubits equal (orders $\times$ candidate DCs) per batch, with no slack variables. The batching scheme holds this at 18 or fewer whatever the total order count, which is what makes the approach possible on current hardware. The cost is that links between batches are lost, so batch composition becomes a quality lever.

7.1 Tested scalability lever

Batching is not proposed on paper — it is built and measured. The 90 highest-value focus orders are split into 15 batches by shared candidate-DC affinity, which keeps each sub-problem at 9–18 qubits while covering \$18.6M of revenue. The cost of batching is visible in the classical results: solving 472 orders together gives 66 moves, whereas the same orders solved separately would lose exactly the connected moves that produce the freight saving in Section 5.

8. Limitations

1. **No demand forecast in the pack.** Nothing holds stock back at the receiving DC for its own future orders, so all our methods move orders more freely than the reference model, which recommended only 3 moves. The 50% reserve sensitivity in Section 6 is the closest available substitute and reduces moves from 41 to 35.
2. **No throughput limit.** The file reports usage only. We use peak observed usage as the limit and a fully booked dock day as the focus-list trigger. Both stand in for a documented rule we cannot compute exactly.
3. **No holiday calendar.** Only weekends can be excluded from revised ship dates.
4. **Two DCs (5083, 5773) have no dock rows**, so the dock rule cannot be applied there. Treating them as zero would wrongly block every move into them.
5. **The quantum results cover 75 orders**, not the full focus set, and match rather than beat the classical heuristic on those batches. Batching also discards the links between batches — which is exactly the coupling that produced the classical solver’s freight saving.

9. Reproducibility

Six notebooks run in order: 01 data exploration, 02 data cleaning, 03 baseline, 04 greedy, 05 classical, 06 comparison, plus the quantum notebooks. Notebook 02 is the only place the model exists. It writes `dom_model.py`, which holds the objective, all seven constraints in both an imperative form (used by the baseline and greedy) and a declarative PuLP form (used by the MILP), the shared gate threshold, the shared big- M , and the scorecard. Notebooks 03, 04 and 05 import that file and add only a search strategy or a solver setting. None of them writes an equation. Notebook 06 reads result files and computes nothing. This structure is what makes the comparison fair rather than merely claimed.

10. Conclusion

Writing DOM as a deterministic binary optimization model and solving it exactly is both possible and worthwhile on this data. The full 472-order problem reaches the proven best answer in about a minute and delivers \$1.93M of extra objective over the current default assignment, with a higher fill rate, 62% less penalty, and less extra freight than the cheaper heuristic pays. The greedy heuristic is still useful for a quick refresh during the day, but it captures under a quarter of the available gain and gets worse as the problem grows. The quantum path shows a clean slack-free encoding, validated against brute-force enumeration and stable across repeated QAOA runs, which is verifiably correct at current qubit counts without yet showing an advantage. The most valuable next step is not a better solver. It is the missing forecast file, which currently makes every method move more orders than a Nestlé planner would accept.